Stockify

Deployment & Technology Revision Guide

Purpose: A quick reference for reopening the project after months and revising its architecture, technologies, deployment settings, important files, environment variables, Git lessons, and current test status.

1. Project at a Glance

Layer	Technology / Service	Role
Landing / Auth UI	React.js	Public website, signup and login
Dashboard	React.js	Trading dashboard
HTTP / API	Axios + Fetch	Frontend ↔ backend communication
Backend	Node.js + Express	REST API and server logic
Database	MongoDB Atlas	Persistent application data
Authentication	JWT	Authentication tokens
Hosting	Netlify + Render	Frontend/dashboard + backend
Source control	Git + GitHub	Version control and deployment source

2. Production Architecture

Flow: User → Netlify Landing Page → Login/Signup → Render API → MongoDB Atlas → Netlify Dashboard.

Live URLs:
Landing: https://stockifyy-frontend.netlify.app
Dashboard: https://stockify-dashboard.netlify.app/
Backend: https://stockify-backend-1cud.onrender.com/

The landing page uses **REACT_APP_DASHBOARD_URL** to reach the deployed dashboard. The dashboard uses **REACT_APP_API_URL** to reach the Render backend.

3. Frontend Technologies

React.js — Component-based UI library; hooks such as useState/useEffect handle state and lifecycle work.

React Router — Client-side routes for dashboard pages such as orders, holdings, positions, funds and apps.

Axios — HTTP client used for backend GET/POST requests.

Fetch API — Also used by some dashboard components.

Environment variables — Create React App variables use the REACT_APP_ prefix and are injected during build.

4. Backend Technologies

Node.js — JavaScript runtime for the backend.

Express.js — Web framework for REST endpoints and middleware.

MongoDB Atlas — Cloud MongoDB database; connection comes from MONGO_URL.

JWT — Authentication token mechanism; secret comes from JWT_SECRET.

CORS — Controls which frontend origins can call the API.

5. Important Project Structure

`backend/` — Node/Express server, models and API logic.
`dashboard/` — authenticated trading dashboard React app.
`frontend/` — public landing page, signup and login React app.

Dashboard files to remember

• dashboard/src/components/Dashboard.js — dashboard shell and React Router routes.

• dashboard/src/components/GeneralContext.js — global buy-window provider/context.

• dashboard/src/components/WatchList.js — watchlist and Buy/Sell actions.

• dashboard/src/components/BuyActionWindow.js — buy UI and POST /addOrders.

• dashboard/src/components/Holdings.js — GET /allholdings.

• dashboard/src/components/Positions.js — GET /allpositions.

Frontend files to remember

• frontend/src/landing_page/home/ — landing/home UI.

• frontend/src/landing_page/signup/SignUp.js — signup.

• frontend/src/landing_page/login/Login.js — login.

• frontend/src/config/api.js — API configuration.

• frontend/netlify.toml — Netlify configuration.

• frontend/public/_redirects — SPA routing support.

6. Environment Variables

Variable	Where	Purpose
MONGO_URL	Render	MongoDB connection
JWT_SECRET	Render	JWT secret
CLIENT_ORIGIN	Render	Allowed frontend origin(s)
REACT_APP_API_URL	Netlify dashboard	Backend URL
REACT_APP_DASHBOARD_URL	Netlify landing	Dashboard URL

Security: Never commit real .env files, MongoDB passwords, JWT secrets, or other credentials to GitHub. Production secrets belong in Render/Netlify environment settings.

7. Netlify Settings

stockifyy-frontend: repository Stockify_trading; branch main; base directory `frontend`; build `npm run build`; publish `build`.

stockify-dashboard: repository Stockify_trading; branch main; base directory `dashboard`; build `npm run build`; publish `build`.

React environment-variable changes require a new Netlify deploy because values are included at build time.

8. Render Backend

The Node/Express backend runs on Render. MongoDB Atlas provides persistence. CORS is configured using the backend environment. After changing backend environment variables, restart/redeploy the service.

9. Git Repository Lesson

The project originally had **frontend** tracked as a Git submodule/embedded repository. Netlify failed with a submodule error. The fix was to remove the submodule entry and commit the actual frontend files into the main repository.

The main repository is **Stockify_trading**. The final repository should contain actual `frontend/` files rather than an unintended submodule.

Useful commands

```
git status
git branch -vv
git log --oneline --decorate -5
git ls-files --stage frontend
git add .
git commit -m "your message"
git push origin main
```

10. Production Verification

Check	Status
Landing page	PASS
Signup + database	PASS
Login	PASS
Dashboard redirect	PASS
Dashboard loads	PASS
Holdings	PASS
Orders	PASS
Positions	PASS
Funds	PASS
Watchlist	PASS
Dashboard ↔ Render API	PASS
Buy option	KNOWN ISSUE — revisit later

11. Coming Back After Months

1. Read the architecture: React → Express → MongoDB.
2. Check the three live URLs to identify which service is failing.
3. Verify Netlify base directory/build/publish settings.
4. Check environment variables before changing code.
5. Check Render logs for API/backend errors.
6. Run git status before edits.
7. Remember React environment variables require a redeploy after changes.
8. Revisit GeneralContext.js, WatchList.js and BuyActionWindow.js for the outstanding Buy issue.

This guide deliberately excludes real credentials and secrets.